

Software Architecture

Exercise – Gaming Platform (iteration #2)

BSc



Ingo Arnold

Copyright Notice

© Ingo Arnold. All rights reserved.

You may use these materials for your personal internal reference purposes only.

You may not reuse these materials in creating your own educational offerings or in your own educational situations like courses, lectures, or seminars nor may you distribute, transfer, resell or otherwise make them available to any other person or party without the expressed permission of the author.

URLs or other references to Internet Web sites in the training materials are subject to change without notice. Unless otherwise indicated, all companies, organisations, domain names, email addresses, persons, places and events depicted in the Training Materials are for illustrative purposes only and are fictitious. No real connection is intended or inferred.

These training materials are provided “as is” and the author makes no warranties, express or implied, with respect to these training materials.

Exercise Opening

Motivation

A new requirement has recently been introduced by a stakeholder group. You are expected to incorporate the requirement and respective responses into the architecture.

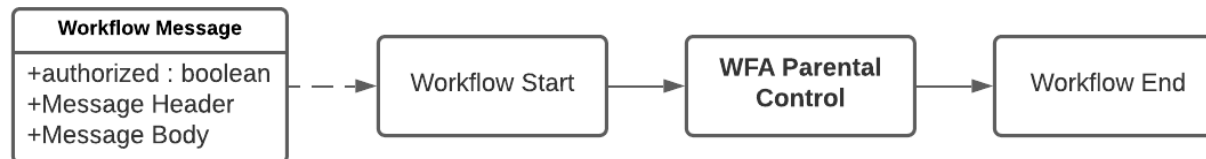
- The use case **add game to platform** was re-discussed in depth and the *quality assurance* stakeholder group requested to go through a comprehensive authorization process whenever new games are added to the platform.
- Quality Assurance wants to ensure that different quality concerns (e.g., parental controls, accessibility, non-maliciousness of the code base) are mandatorily reviewed by the appropriate departments.
- The requirements for the authorization process were so extensive that it was already decided to establish the capabilities of a *workflow engine*. This allows authorization processes to be flexibly changed (i.e., reconfigured) – that means: without going through recompilation cycles.
- The workflow engine requirements are outlined in more detail on the subsequent slides

Exercise Opening

Motivation

A new requirement has recently been introduced by a stakeholder group. You are expected to incorporate the requirement and respective responses into the architecture.

- A workflow comprises ...
 - A set of workflow activities (WFA)
 - Connections between workflow activities (chaining activities unidirectionally)
 - An initial message or request (Workflow Message) that is populated into the first workflow activity and then propagated through the entire workflow (i.e. activities following the first one in the chain) through to its end
- In the simple workflow I illustrated below, a *workflow message* is populated into the component *workflow start*. Next the message is reviewed by a single workflow activity (*WFA Parental Control*), before it is finally processed by the *workflow end* component.

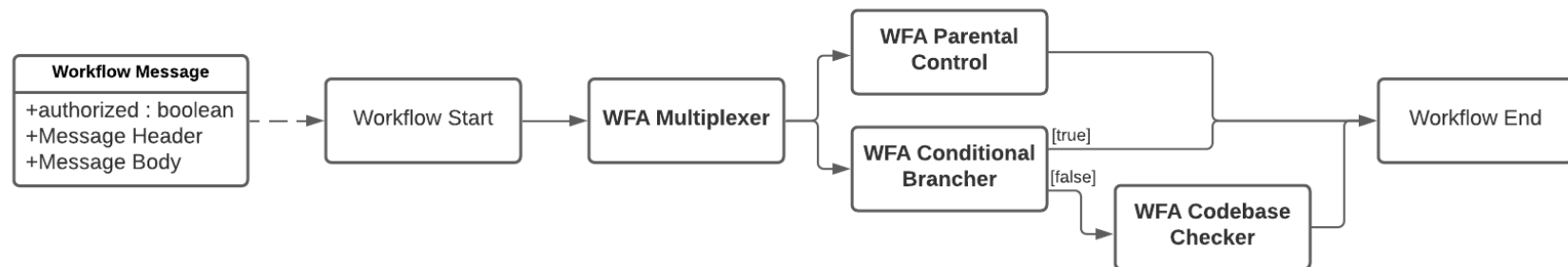


Exercise Opening

Motivation

A new requirement has recently been introduced by a stakeholder group. You are expected to incorporate the requirement and respective responses into the architecture.

- In a more complex workflow configuration a message (*workflow message*) is populated into *workflow start* from which it is further handed into a *WFA multiplexer* component. A multiplexer broadcasts the message into all connected components. In this case these are *WFA parental control* and *WFA conditional brancher*. Conditional brancher makes a decision (based on defined logical reasoning) and forwards the message to either *WFA codebase checker* (if the result of the logical reasoning was *false*) or right into the *workflow end* component. *WFA parental control* hands its processed message over to *workflow end*.



Exercise Opening

Motivation

A new requirement has recently been introduced by a stakeholder group. You are expected to incorporate the requirement and respective responses into the architecture.

- The workflow framework shall provide a set of WFA types which enable the composition of workflows in a very flexible and reusable manner.
- All WFA types have the following characteristics in common:
 - A WFA component maintains connections to at least one subsequent component (e.g., *WFA parental control* maintains exactly one connection, *WFA conditional brancher* maintains two connections, while *WFA multiplexer* maintains many connections).
 - A WFA component receives *workflow messages*, processes them, and propagates the processed message to its connected WFA components for further massaging.
 - Any WFA component receives its input from either a *workflow start* or from a respective WFA component (i.e., its predecessor in the workflow chain).
 - Irrespective of the concrete work a WFA component performs on its message input, the output is propagated to either the final receiver (*workflow end*) or to the next WFA component in the chain.
 - Workflow messages are comprised of a *header* and a *body* portion. They further maintain an *authorized* flag, which is toggled to *false* if a WFA component decides to veto its workflow approval.

Exercise Opening

Motivation

A new requirement has recently been introduced by a stakeholder group. You are expected to incorporate the requirement and respective responses into the architecture.

- In an initial version of the workflow framework the following WFA component types establish the framework:
 - **WFA multiplexer** components broadcast received messages (e.g., based on a particular rule) to all registered successor WFA components.
 - **WFA parental control** components validate received messages regarding G-rated content and propagate the transformed message into their registered (singular) successor.
 - **WFA code base checker** components validate received messages regarding a non-malicious code base and propagate the transformed message into their registered (singular) successor.
 - **WFA conditional brancher** components have 2 outgoing WFA component branches (i.e., a *true* and a *false* branch). A defined branching algorithm is applied to data in the message header in order to make a branching decision and to determine which of both successors receives the message for further processing.
 - **Workflow messages** contain a set of name-value pairs as header attributes while the respective game is included in their body portions.

Exercise Opening

Motivation

A new requirement has recently been introduced by a stakeholder group. You are expected to incorporate the requirement and respective responses into the architecture.

- The *workflow start* component acts as a workflow configurator and passes a received message into the first component in its particular workflow.
- The *workflow end* component may receive messages from several predecessor components. It evaluates all messages and calculates an overall authorization result. Once a workflow is completed, *workflow end* components can be queried to check whether the workflow was authorized in its entirety.

Exercise Opening

Motivation

Incorporate the new requirements and specify them appropriately. Distinguish between use case specific and system-wide requirements, constraints and assumptions.

Analyze the new requirements and **create an analysis model** of appropriate component candidates that highlights their static and dynamic relationships, such that they implement the desired **Workflow Framework** feature.

Create the complementary facets of the system architecture in the form of an architecture approach. Clarify component interfaces and specifically consider patterns that might come in handy in implementing the new requirements.

Finally, **implement your design in code** and assemble the individual parts into an overarching solution.

Requirements Elicitation Scenario

The **specification** of a **scenario** that reflects the desired **Workflow Framework** feature has **already been elaborated**.

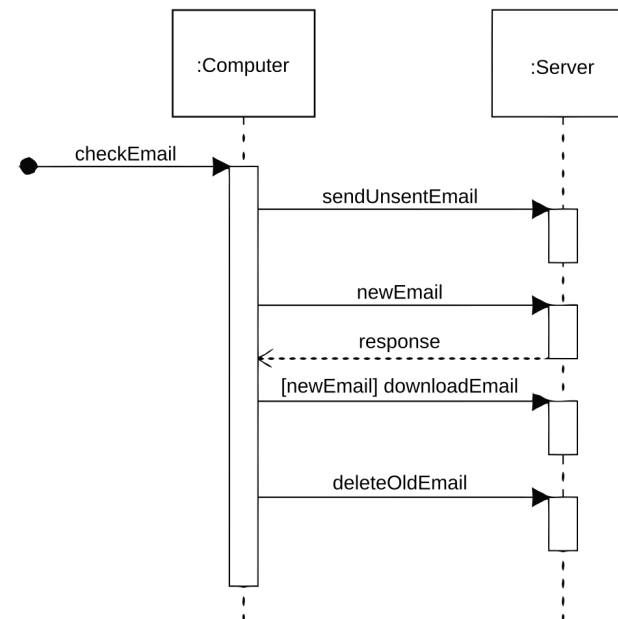
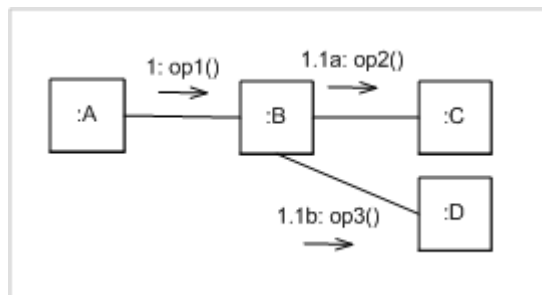
1. A *chess* game is created and included in a workflow message body.
2. The message header is set to the name of the game.
3. A WFA parental control component is connected to WFA multiplexer component.
4. The workflow end component is connected to the WFA parental control component.
5. The workflow end component is also connected to a WFA conditional brancher (true-branch).
6. A WFA codebase checker component is connected to the same WFA conditional brancher (false-branch) and the workflow end component is connected to the WFA codebase checker.
7. The WFA conditional brancher is finally connected to WFA multiplexer.
8. WFA multiplexer (as the first WFA component) is registered with workflow start.
9. The message is populated to workflow start and propagated further through the entire workflow. Each individual WFA component performs its duties on the received message and sets the authorized flag, accordingly.
10. Workflow end receives multiple messages (one from each of its predecessors) and calculates the overall authorized flag.
11. Workflow end is queried for whether the message was accepted or rejected by the entire workflow.

System Architecture

Scenario → Logical View

Create the scenario in the form of interacting objects (UML communication diagram).

These are two examples of communication diagrams in UML. What specific purpose does each type of communication diagram pursue?



System Architecture

Scenario → Logical View

When you describe the Workflow Framework scenario in the form of interacting objects, distinguish between (a) creating a specific workflow configuration or instance (steps 1-8), and (b) populating a message into the workflow (steps 9-11).

Use the Miro-Board shared via e-Mail to create the Logical View.



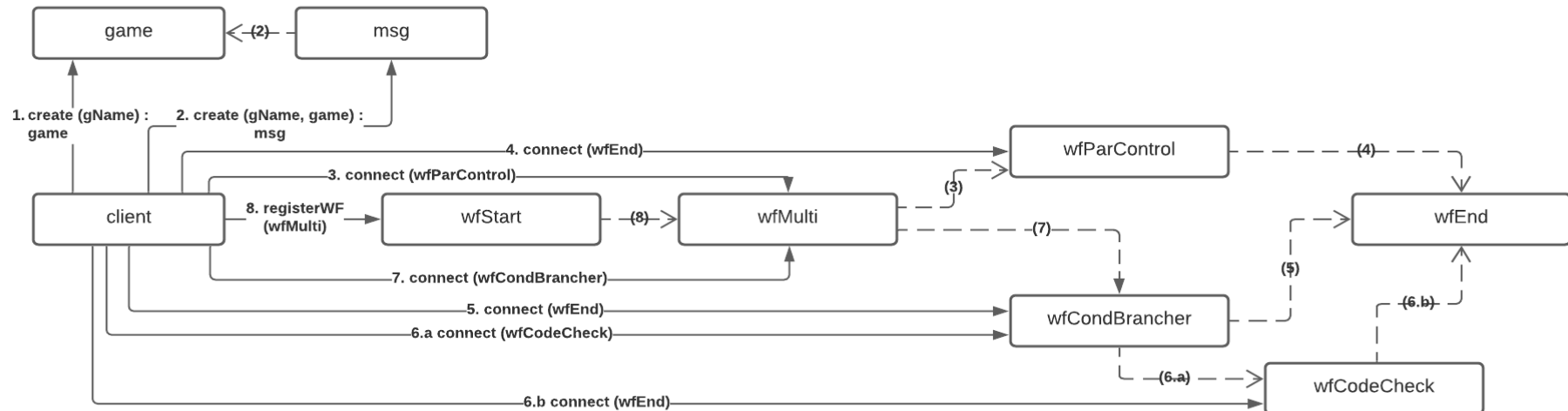
example

System Architecture

Scenario → Logical View

Create the scenario in the form of interacting objects (communication diagram).

- Building and registering the workflow with the workflow start component.





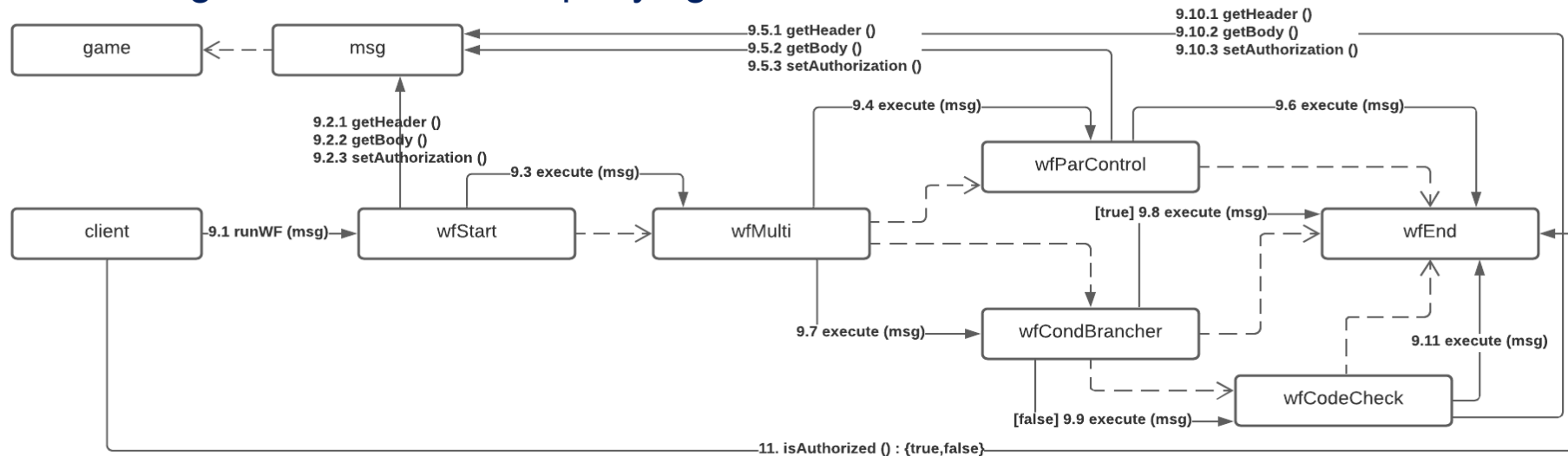
example

System Architecture

Scenario → Logical View

Create the scenario in the form of interacting objects (communication diagram).

■ Performing the workflow and querying authorization status.



System Architecture

Logical View → Development View

Systematically develop an **object-oriented design** in the **development view** for which you perform the steps, below.

1. Create a list of all classes to be programmed, so that ...
 - each object can be instantiated by exactly one class
 - identically behaving objects are instantiated by a common class
 - each class instantiates at least one of the required objects
2. Identify details for each of the identified classes:
 - Each arrow arriving at an object (e.g., in a sequence diagram) represents the activation of a method.
 - For these you provide a callable (public) method in the corresponding class.
 - The result of this step are complete method signatures including return value and parameters as well as additional information about method behaviour.
 - This means at this stage you also consider which information is passed between objects.

System Architecture

Logical View → Development View

Create a list of all classes to be programmed ...

Class	Object(s) / Class(es)

... and determine methods, method signatures, and relationships between identified classes.

Use the Miro-Board shared via e-Mail to create the Development View.



example

System Architecture

Logical View → Development View

Create a list of all classes to be programmed ...

Class	Object(s)
Game	game
WfMessage	msg
WfStart	wfStart
WfMultiplexer	wfMulti
WfParControl	wfParControl
WfEnd	wfEnd
WfCondBrancher	wfCondBrancher
WfCodeCheck	wfCodeCheck



example

System Architecture

Logical View → Development View

Create a list of all classes to be programmed ...

Class	Object(s)
Game	game
WfMessage	msg
WfStart	wfStart
WfMultiplexer	wfMulti
WfParControl	wfParControl
WfEnd	wfEnd
WfCondBrancher	wfCondBrancher
WfCodeCheck	wfCodeCheck
WfBrancher	super type for WfCondBrancher
WfBroadcaster	super type for WfMultiplexer
WfTask	super type for WfCodeCheck, WfParControl
WfBuildingBlock	super type for WfBroadcaster, WfBrancher, WfTask

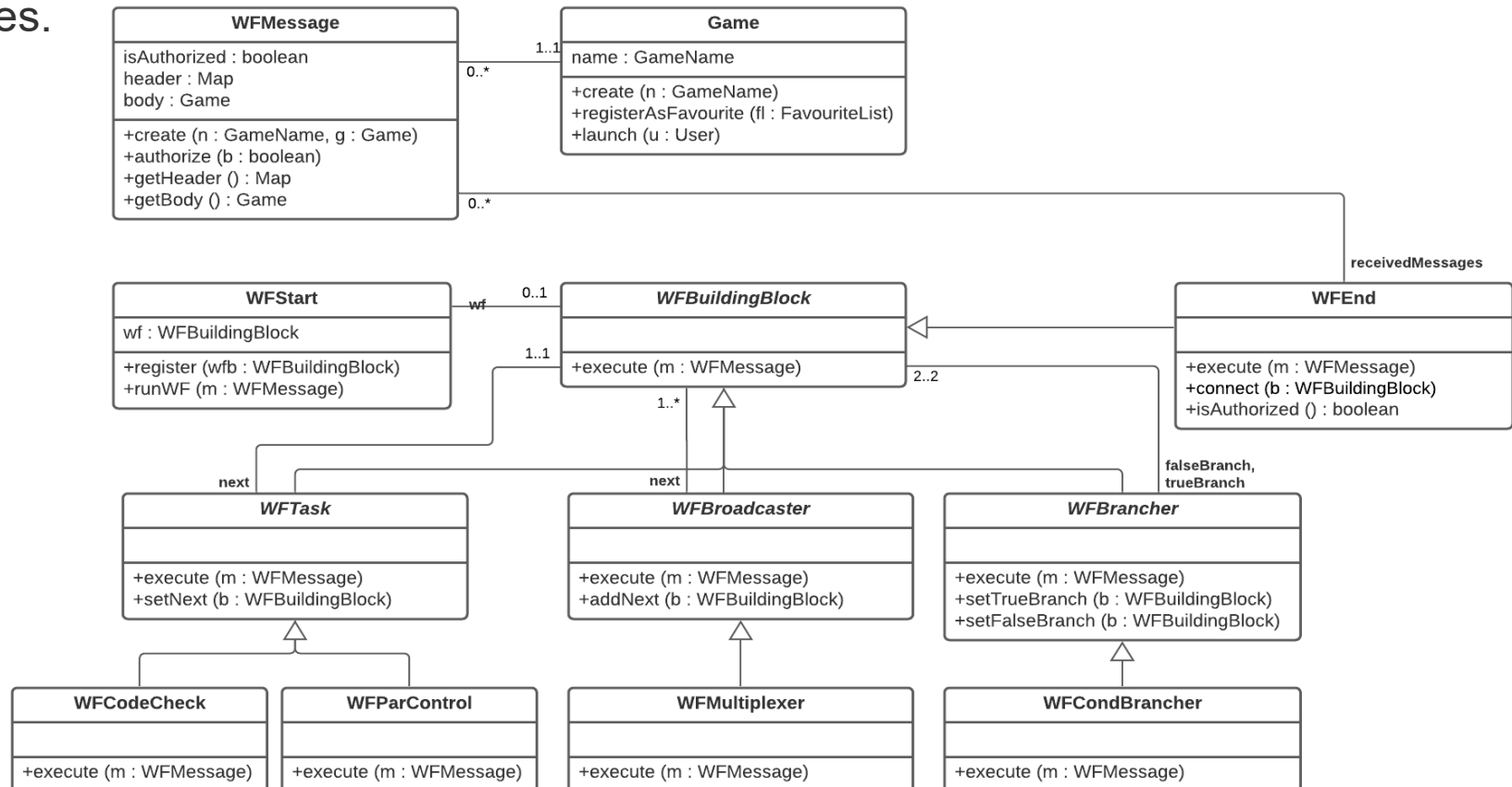


example

System Architecture

Logical View → Development View

Determine methods, method signatures, and relationships between identified classes.



System Implementation

Code Model in Java

Finally, you build on the architecture design and implement an initial version of the Workflow Framework in Java.

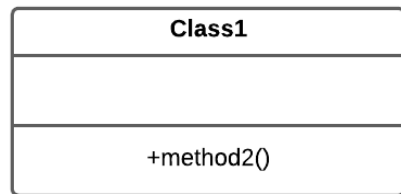
Use the scenario that was given above (slide 10) as an initial test case for the Workflow Framework.

System Implementation

Code Model in Java

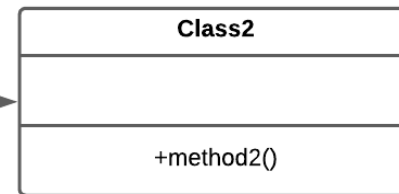
To convert the development view into a corresponding Java programme, we follow the following systematic. Classes that have a use relationship never bind directly to each other, but always indirectly via a corresponding interface.

Wrong



```
public class Class1 {
    Class2 refObject2 = new Class2();

    public void method1() {
        refObject2.method2();
    }
}
```



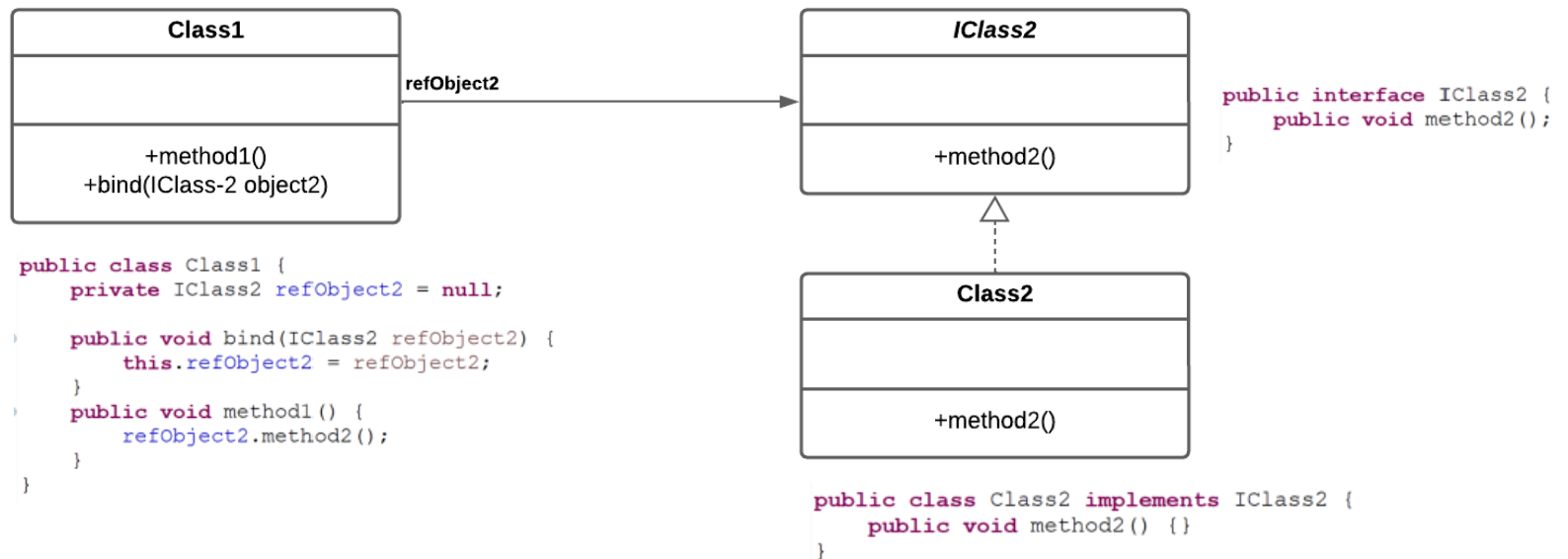
```
public class Class2 {
    public void method2() {}
}
```

System Implementation

Code Model in Java

To convert the development view into a corresponding Java programme, we follow the following systematic. Classes that have a use relationship never bind directly to each other, but always indirectly via a corresponding interface.

Correct



System Implementation

Code Model in Java

To convert the development view into a corresponding Java programme, we follow the following systematic. Classes that have a use relationship never bind directly to each other, but always indirectly via a corresponding interface.

Correct

To execute the entire programme, we will later create corresponding objects outside each implementation class (in a dedicate Composer class) and connect all objects according to our design.

```
public class Composer {  
    public static void main(String [] args) {  
        Class1 c1 = new Class1();  
        Class2 c2 = new Class2();  
  
        c1.bind(c2);  
        c1.method1();  
    }  
}
```



example

System Implementation

Code Model in Java



Wrap-Up

■ Motivation and New Requirement

- A **new stakeholder requirement** introduces an **authorization workflow** for adding games to the platform.
- **Goal: ensure quality assurance (QA) through a structured review** of new games **before release**.
- Quality concerns addressed:
 - Parental controls (age appropriateness)
 - Code safety (non-malicious codebase)
- **Requirement** leads to the **design** and implementation of a **Workflow Framework**, enabling **flexible** and **reconfigurable authorization processes**.
- **Flexibility** is achieved **via a workflow engine: workflow logic** can be **changed at runtime without recompilation**.

Wrap-Up

■ Workflow Framework Overview

- **Core Concept: workflows consist of interconnected Workflow Activities (WFAs) processing Workflow Messages (WFMs).**
- **A Workflow Message:**
 - Contains a **header** (metadata) and **body** (e.g., game details).
 - Has an *authorized* flag toggled to **false** if any **WFA** vetoes approval.

■ Basic Workflow Structure

- **Workflow Start initializes message propagation.**
- **WFA Components process and forward messages:**
 - **Multiplexer** → broadcasts message to multiple successors.
 - **Parental Control** → checks age-appropriate content.
 - **Codebase Checker** → validates non-malicious code.
 - **Conditional Brancher** → makes decisions (true/false path).
- **Workflow End aggregates results and determines overall authorization status.**

Wrap-Up

■ Logical View

- **Logical view** models **interacting objects** (e.g., based on a communication diagram).
- **Two perspectives:**
 - Workflow Construction – instantiating and linking components.
 - Workflow Execution – message propagation and authorization evaluation.
- **Each object corresponds** to one **WFA type** or **workflow element**.
- **Purpose:** understand **dynamic message flow** and **collaboration structure**.

Wrap-Up

■ Development View

- **Transition** from **object interactions (logical view)** to **class structure (development view)**.
- **Identified classes** (example list):
 - Game, WFMessage, WFStart, WFMultiplexer, WFPControl, WFEnd, WFCondBrancher, WFCCodeCheck
 - Abstract super classes:
 - WFBuildingBlock (root)
 - WFBroadcaster (for Multiplexer)
 - WFBrancher (for Conditional Brancher)
 - WFTask (for Parental Control, Code Checker)
- **Each class defines ...**
 - **public methods corresponding** to **message-handling interactions**.
 - **relationships** (composition, inheritance, and associations).
- **Class hierarchy promotes reuse, extensibility, and clear abstraction of workflow logic.**

Wrap-Up

■ Development View

- **Transition** from **object interactions (logical view)** to **class structure (development view)**.
- **Identified classes** (example list):
 - Game, WFMessage, WFStart, WFMultiplexer, WFPARControl, WFEnd, WFCondBrancher, WFCodeCheck
 - Abstract super classes:
 - WFBuildingBlock (root)
 - WFBroadcaster (for Multiplexer)
 - WFBrancher (for Conditional Brancher)
 - WFTask (for Parental Control, Code Checker)
- **Each class defines ...**
 - **public methods corresponding** to **message-handling interactions**.
 - **relationships** (composition, inheritance, and associations).
- **Class hierarchy promotes reuse, extensibility, and clear abstraction of workflow logic.**

Wrap-Up

■ Overall Learning Objectives

- You should be able to ...
 - **Explain** the **motivation behind introducing a workflow framework**.
 - **Model workflow interactions** (logical view) and **derive corresponding class structures** (development view).
 - **Identify** and **justify** appropriate **object-oriented design mechanisms** (inheritance, interfaces, abstraction).
 - **Implement** the resulting solution design (class and communication diagrams) in Java.

Questions

